

Achilles LoRA Eğitimi — Detaylı Anlatım

Sürüm: v1.2 · 2026-06-17

Sürüm Geçmişi

Sürüm	Tarih	Değişiklik
v1.0	2026-06-16	İlk kapsamlı sürüm
v1.1	2026-06-16	Denetçi düzeltmeleri: <code>check_flags</code> fiili davranışı, <code>cloud_notebook.py</code> net durumu, <code>chunk_size=1200</code> kesinleştirildi, <code>classify_curriculum</code> imzası iki-parametrelili olarak düzeltildi
v1.2	2026-06-17	İleri LoRA teknikleri entegrasyonu (araştırma turu): rsLoRA / DoRA / <code>init_lora_weights</code> (PiSSA/OLoRA/EVA/LoftQ) / LoRA+ / NEFTune + regularizasyon (<code>warmup-cosine-weight-decay-grad-clip</code>) PEFT trainer'a saf, offline-test edilebilir builder'larla bağlandı; <code>discipline_safe</code> profili (v5 catastrophic-forgetting reçetesi); bulut notebook parametrik (<code>alpha/dropout/rsLoRA/NEFTune</code>); degenerasyon tespiti n-gram/satır döngüsünü de yakalar. Yeni: Aşama 8 + Araştırma Kaynakları & Log .

Not: Yeni eğitim geliştirmesinde sürüm numarası artırılır ve değişiklik buraya eklenir.

Amaç ve Kapsam

Bu doküman, Achilles araştırma sisteminin **LoRA (Low-Rank Adaptation) eğitim hattını** uçtan uca anlatır. Hat, akademik PDF literatüründen başlayıp, denetlenmiş veri üretimine, kalite kapılarına, yerel/bulut eğitime, değerlendirmeye ve nihayet üretime terfi eden bir adapter'a kadar uzanır.

Kapsam:

- Veri üretimi** — iki bağımsız dataset builder (bilgi kartı + sentetik QA), RAFT/grounding, reasoning-chain verisi, birleştirme (unified), yakın-tekrar (near-duplicate) dedup ve müfredat seviyeleri (0–4).
- Kalite kapıları** — Gate 0–8, güvenlik tarayıcısı (BLOCKER) ve kontrol düzlemi (control plane) durum makinesi.
- Eğitim backend'leri** — MLX vs PEFT platform tespiti, dry-run vs `--run`, detached (tek-tık) süreç.
- LoRA profilleri** — `lora_profiles.yaml` ön-ayarları.
- Değerlendirme** — `adapter_eval` ile gerçek PEFT yükleme, base-vs-adapter yan-yanaya kıyas, v5 regresyon dersi, negasyon-kör uyarısı.
- Adapter registry yaşam döngüsü** — candidate → ... → production, kullanıcı onayı zorunluluğu.
- Bulut + aşamalı eğitim** — Kaggle/Colab, Stage 1/2, GGUF + Modelfile → Ollama.

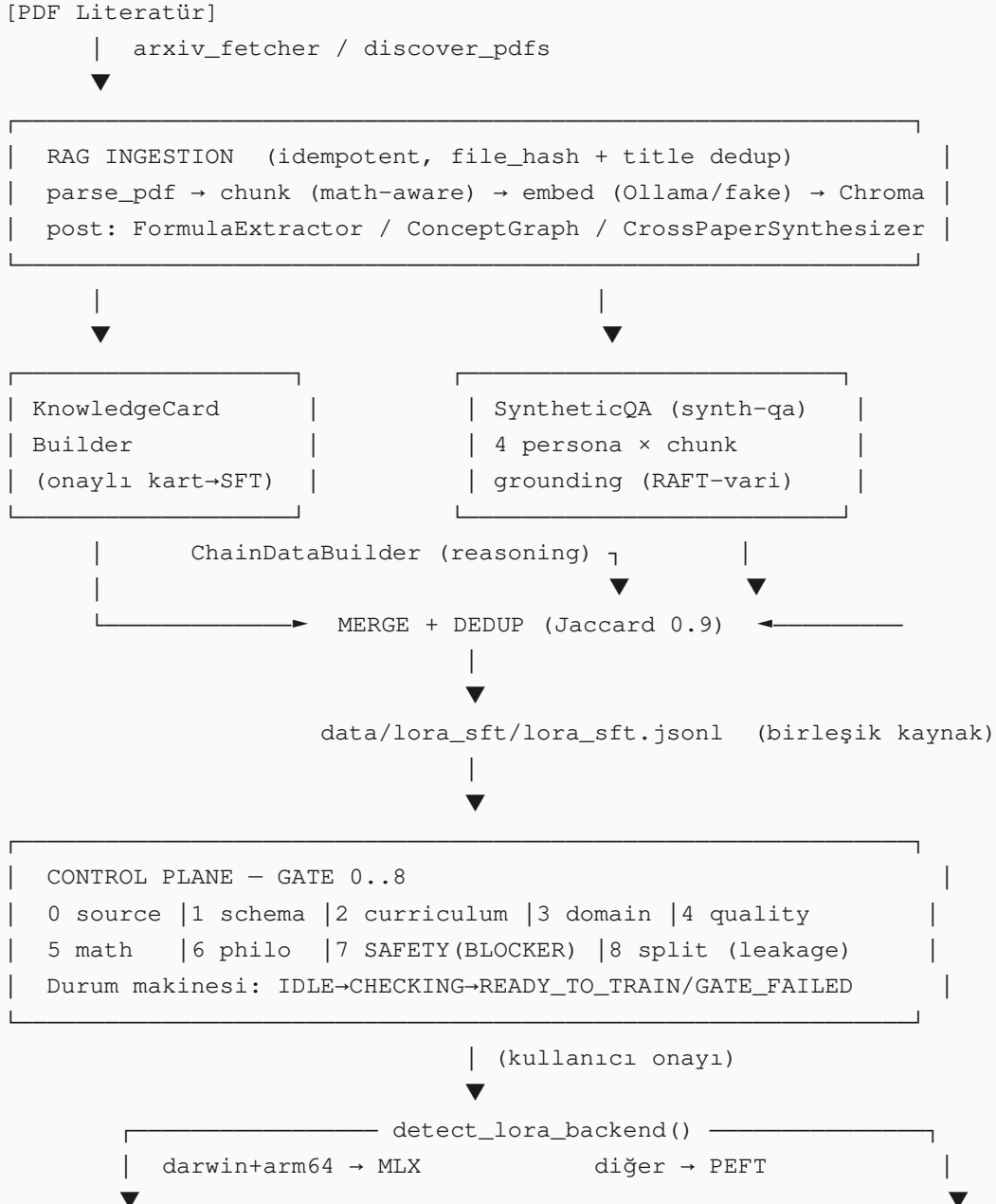
8. Loop'lar ve otomasyon — LoRA ile ilgili tüm döngüler.

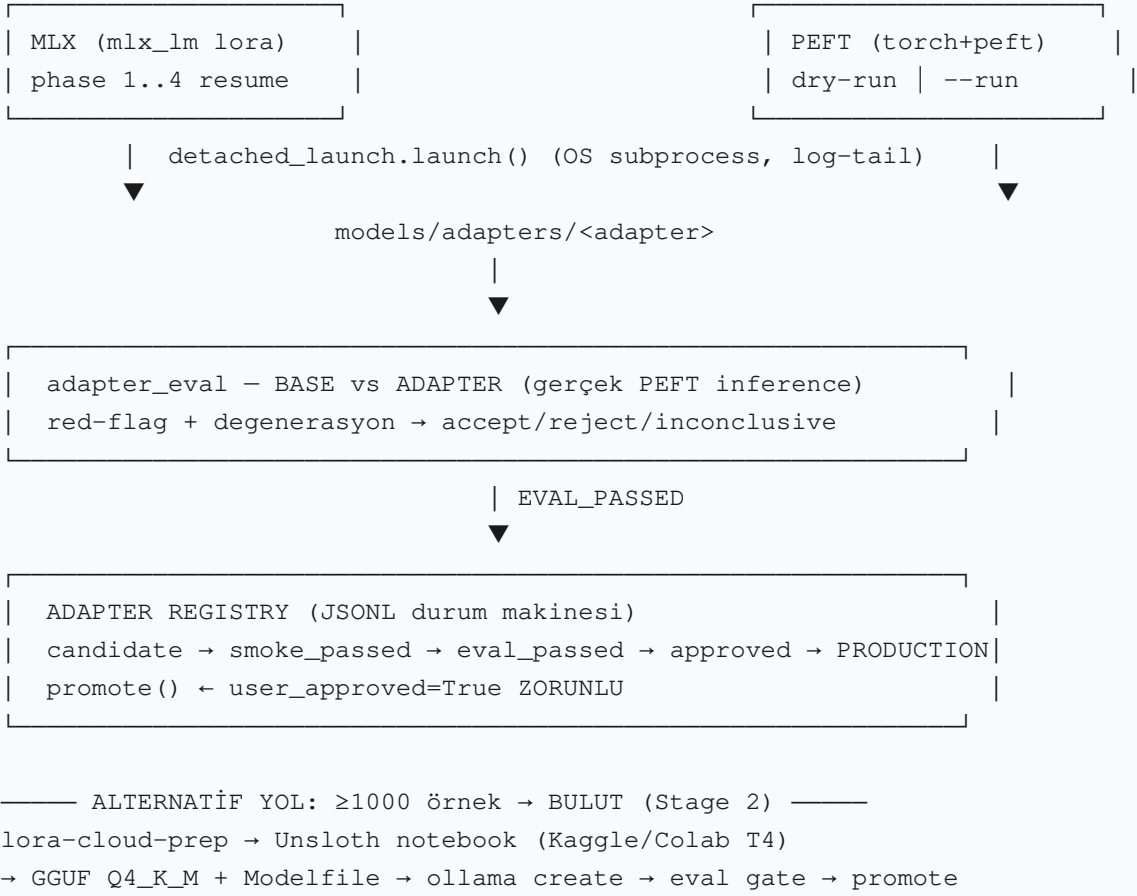
Bu doküman, CLAUDE.md kurallarına bağlıdır: kodda bulunamayan özellikler açıkça "kodda bulunamadı" olarak işaretlenir; hiçbir bileşen test edilmeden "çalışıyor" denmez; iddialar mümkün olduğunca `dosya:satır` ile desteklenir.

Önemli uyarı (durum): `storage/auto_lora_state.json` örneğinde sistemin son durumu `train_failed` ("Eğitim COMPLETED olmadı"), Gate sonucu ise `passed` görünmektedir. Yani kalite kapıları geçilmiş ancak eğitim adımı tamamlanamamıştır. Doküman, hattın **tasarımını** anlatır; hattın baştan sona başarıyla koştuğu **iddia edilmez**.

Mimari Genel Bakış

ACHILLES LoRA EĞİTİM HATTI





Aşama Aşama Yöntemler

Aşama 0 — RAG Ingestion (veri kaynağının hazırlanması)

LoRA verisi, doğrudan PDF korpusundan türeyen yapılandırılmış bilgiye dayanır. Bu nedenle ingestion, eğitim hattının "sıfır" aşamasıdır.

- **NE:** PDF → metin → matematik-korumalı chunk → embedding → Chroma + SQLite; ardından formül/kavram/sentez post-processing.
- **NASIL:** `PaperIndexer.ingest_one()` idempotent çalışır: önce `file_hash` (aynı bayt → skip), sonra normalize edilmiş başlık (`find_paper_by_title`) ile çift-kayıt engellenir. Chunking, `$$...$`, `\[...]`, `\begin{equation}` ve `%12+` matematiksel karakter oranı olan paragrafları **bölmez** (`_is_math_heavy`).
- **DOSYA:** `app/memory/paper_indexer.py:63-215`, `app/ingestion/chunker.py:23-156`, `app/ingestion/pdf_parser.py:56-73`, `app/ingestion/arxiv_fetcher.py:48-164`.
- **MODEL/KÜTÜPHANE:** PyMuPDF/pypdf (PDF), Ollama `nomic-embed-text` (embedding; offline'da deterministik SHA256 fake fallback), ChromaDB (HNSW cosine), SQLAlchemy 2.0 (SQLite).
- **PARAMETRELER:** `chunk_size=1200` (kesin varsayılan, `app/config/settings.py:77`), `chunk_overlap=200` (`settings.py:78`); chunker bu değerleri `settings.chunk_size`'tan alır (`app/ingestion/chunker.py:85`). `rag_contextual_embed` (varsayılan kapalı). Not: `synthetic_qa_builder` içindeki `_MAX_PASSAGE_CHARS=1800`, prompt'a gömülen pasaj üst sınırıdır; `chunk_size` ile karıştırılmamalıdır.

Doğrudan LoRA verisi üretmediği için bu aşama eğitim hattının ön-koşuludur; detayı RAG dokümanında genişler. Burada yalnızca formül/kavram/sentez çıktısının `training_examples` tablosuna yazıldığını not ederiz (`CrossPaperSynthesizer.synthesize_all()`).

Aşama 1 — Veri Üretimi (iki dataset builder)

1a. Bilgi Kartı Dataset Builder

- **NE:** SQLite `knowledge_cards` tablosundan onaylı kartları (`review_status=approved AND lora_eligible=1`) OpenAI-uyumlu `messages` SFT örneğine çevirir.
- **NASIL:** `card_to_lora_example()` filtreyi uygular (`onaysız/uygunsuz → None`); `_build_answer()` özet + ana iddia + trading geçerliliği + hipotezler + formülleri birleştirir. Metadata: `source_id` (`dedup/split` için), `difficulty` [0,1], `stage` (`lora_phase_1..4`).
- **DOSYA:** `app/lora/dataset_builder.py:34-114`.
- **MODEL/KÜTÜPHANE:** Saf Python + SQLAlchemy; bu aşamada LLM yok (kart üretimi zaten yapılmış sayılır).
- **NOT:** Kartların kendisi `KnowledgeCardBuilder.build()` ile üretilir (LLM, temperature 0.1, Ollama JSON modu, retry boş `main_claim`'de; `app/brain/knowledge_card_builder.py:138-180`). Güven seviyesi daima "draft" — insan onayı zorunlu.

1b. Sentetik QA Motoru (synth-qa) — RAFT-benzeri grounding

- **NE:** Makale chunk'larından LLM ile çeşitli, **grounded** (yalnız pasaja dayalı) Türkçe QA örnekleri üretir. Hedef: ~15–50 şablondan ~1000–2000 örneğe büyüme.
- **NASIL:** Her chunk bir **persona** alır (kantitatif araştırmacı / risk yöneticisi / backtester / şüpheli denetçi — `PERSONAS`, `synthetic_qa_builder.py:34-51`), rotasyonla. `build_for_chunk()` prompt kurar, deterministik seed (`eff_seed = seed + persona_index`), temperature=0.4, fmt="json", max_tokens=1600. JSON parse → normalize → **grounding** üç-kapı:
 1. Cevaptaki anlamlı sayılar (`_SIG_NUM_RE = \d+[.,]\d+|\d+%|\d{2,}`) pasajda geçmeli;
 2. bilgi-fakir pasaj reddi;
 3. en az 1 anchor örtüşmesi. Ardından uzunluk filtresi (`min_answer_chars=60`).
- **DOSYA:** `app/brain/synthetic_qa_builder.py:111-138` (`_is_grounding`), :216-238 (`_build_prompt`), :241-317 (`build_for_chunk`), :319-347 (`build_for_paper`).
- **MODEL/KÜTÜPHANE:** Ollama `qwen3:4b` (enjekte edilebilir LLM → çevrimdışı test).
- **PARAMETRELER (CLI):** `synth-qa --per-chunk 5 --max-chunks 12 --max-papers 0 --append --seed 0`; toplu: `synth-qa-bulk --batch 5 --target 1000` (checkpoint'li, çökme-güvenli; tam batch implementasyonu kaynak bulgusunda "tam okunmadı" notuyla **kısmen doğrulandı**).

RAFT notu: Direktifte "RAFT" geçer. Kaynak bulgusu (`lora-dataset`) açıkça belirtir: "**RAFT seed** terimi kodda geçmez; mevcut grounding mekanizması RAFT'ın (Retrieval-Augmented Fine-Tuning) "cevap yalnız verilen pasajdan kaynaklanmalı" ilkesinin uygulamasıdır, ancak ayrı bir RAFT modülü **kodda bulunamadı**. Repo kökünde `storage/_gen_raft_seed.py` adlı izlenmemiş bir dosya görülmektedir (git status); içeriği bu bulgularda yer almadığından işlevi **doğrulanmadı**.

1c. Chain Data Builder (reasoning zinciri)

- **NE:** Araştırma oturumlarını (`research_sessions`) "nasıl düşünmesi gerektiğini" öğreten örneklere çevirir.
- **NASIL:** Prompt = soru + mevcut formüller + kavram grafiği; completion = Düşünce → Önerilen indikatör → Bileşenler → Giriş kuralları → Beklenen avantaj → Başarısızlık koşulları → Backtest (Sharpe/getiri/verdict) → Yansıma → İyileştirme.
- **DOSYA:** `app/research/chain_data_builder.py:96-174`. Çıktı: `data/training/research_chains.jsonl`.

1d. Unified Dataset Builder

- **NE:** Üç kaynağı (kartlar + mastery sınavları + tool-use seansları) tek MLX-formatlı (`{"prompt", "completion"}`) dosyada birleştirir.
- **NASIL:** `DatasetBuilder.collect()` + `MasterySFTBuilder.collect()` + `build_tool_use_dataset()` → `random.Random(seed).shuffle` → yaz.
- **DOSYA:** `app/training/unified_dataset.py:50-108`. Çıktı: `data/training/unified_sft.jsonl`.
- **NOT:** `MasterySFTBuilder` (`app/training/mastery_sft_builder.py`) ve `build_tool_use_dataset()` tanım dosyaları bu bulgularda **okunmadı** → iç davranış doğrulanmadı.

1e. Dedup + Müfredat + Bölme

- **Dedup:** `dedup_jsonl_lines()` — tam içerik SHA256 + yakın-tekrar Jaccard ≥ 0.9 (muhafazakâr, meşru farklılıklar korunur). İlk görülen tutulur. `app/brain/synthetic_qa_builder.py:427-468`.
- **Müfredat (0–4):** `classify_curriculum(card_json, difficulty)` — `LEVEL_0` (0.0–0.2) ... `LEVEL_4` (0.8–1.0). İmza iki konumsal parametre alır (`app/lor/curriculum.py:41`); `card_json` şu an kullanılmaz (`_ = card_json`, içerik-temelli genişletme için ayrılmıştır) ama **zorunludur** — tek argümanlı çağrı (`classify_curriculum(difficulty)`) hata verir. Sınıflandırma deterministik olarak yalnızca `difficulty`'ye dayanır. `app/lor/curriculum.py:41-59`.
- **Bölme (Gate 8 ön-fonksiyonu):** `split_dataset()` — `source_id` (`paper_id`) bazında grupta, `seed=42` shuffle, `train=0.8/valid=0.1/test=0.1`; **aynı makale tek bölmede kalır**. `check_leakage()` kesişimleri kontrol eder. `app/lor/dataset_splitter.py:40-103`.

Aşama 2 — Kalite Kapıları (Gate 0–8) ve Control Plane

- **NE:** Birleşik kaynağı sekiz sıralı kalite kapısından geçirir; temiz eğitim setini üretir ve raporlar. **Ağır eğitim başlatmaz.**
- **NASIL:** `LoRAControlPlane.run_audit()` (Gate 0–7) veya `run_full()` (Gate 0–8) → `PipelineReport`. Gate'ler saf fonksiyonlardır.

Gate	Ad	Kontrol	Tür
0	source	<code>review_status=approved</code> , ≥ 1 domain, <code>created_at</code>	BLOK
1	schema	<code>messages list</code> ≥ 3 ve sıra <code>[system,user,assistant]</code> , <code>metadata</code>	BLOK

2	curriculum	difficulty $\in [0,1]$	BLOK
3	domain	classify_domains() ≥ 1 domain	BLOK
4	quality	uzunluk <50 / soru-cevap örtüşme ≥ 0.9 / SHA256 duplicate	BLOK (temiz çıktı)
5	math	lookahead/survivorship bayrakları (review) + aşırı-emin dil (BLOK)	BLOK+UYAR
6	philosophy	çelişki/korelasyon-nedensellik işareti	UYAR (blok değil)
7	safety	sır/PII/finansal yönlendirme	BLOCKER
8	split	train/valid/test + leakage	BLOK (sızıntıda)

- **DOSYA:** app/lora/gates.py:77-275, app/lora/control_plane.py:90-128, app/lora/quality_filter.py:62-121, app/lora/safety_scanner.py:14-75, app/lora/math_verifier.py:54-126, app/lora/domain_classifier.py:27-168.
- **GÜVENLİK TARAYICISI (Gate 7 / BLOCKER):** scan_for_secrets() regex setiyle: api_key (32+ alfanumerik), private_key (PEM başlığı), wallet_address, credential_assignment, email, phone (TR/int), national_id (11 hane); ayrıca FINANCIAL_DIRECTIVES ("şimdi al", "garanti kar", ...). **Tek ihlal tüm batch'i reddeder** — kısmi geçiş yoktur (safety_scanner.py:56-75, gates.py:244-259).
- **KONTROL DÜZLEMİ DURUM MAKİNESİ:** PipelineStage enum (auto_pipeline.py:27-38): IDLE $\rightarrow$ CHECKING $\rightarrow$ {GATE_FAILED | READY_TO_TRAIN} $\rightarrow$ TRAINING $\rightarrow$ {TRAIN_FAILED | EVALUATING} $\rightarrow$ {EVAL_SKIPPED | EVAL_FAILED | EVAL_PASSED} $\rightarrow$ PROMOTED. Durum storage/ auto_lora_state.json'da kalıcıdır.

Dürüstlük notu: Gate 4 duplicate izleme yalnız **oturum-içidir** (DB'de kalıcı duplicate log **kodda bulunamadı**). Gate 2 yalnızca difficulty aralığını kontrol eder; %60/%30/%10 müfredat pacing **kodda bulunamadı** (yalnız ajan tanımlarında metin olarak geçer).

Aşama 3 — Eğitim Backend'leri

3a. Platform Tespiti

- **NE:** Hangi backend? **NASIL:** detect_lora_backend() — sys.platform=="darwin" and platform.machine()=="arm64" $\rightarrow$ "mlx", aksi $\rightarrow$ "peft". **DOSYA:** app/training/backend.py:9-13. CLI'da --backend auto|mlx|peft (main.py:258).

3b. PEFT Eğitimi (Windows/Linux — bu projenin ana platformu)

- **NE:** torch + transformers + peft ile in-process LoRA eğitimi.
- **NASIL:** dry_run() varsayılan — --run yoksa eğitim **başlatılmaz** (sadece bağımlılık kontrolü + kurulum komutu). train() (--run ile): tokenizer/model yükle $\rightarrow$ LoraConfig uygula $\rightarrow$ tokenize (padding yok, dinamik collator) $\rightarrow$ Trainer.train() $\rightarrow$ loss eğrisi JSON.
- **DOSYA:** app/training/peft_lora_train.py, CLI app/main.py:244-345.
- **MODEL/KÜTÜPHANE:** Qwen/Qwen3-4B-Instruct-2507 (varsayılan base), torch/transformers/

peft (kurulu: peft 0.19.1, transformers 5.12.0).

- **PARAMETRELER:** iterations=300, batch_size=1 (config) / 2 (CLI öneri, 8GB), lr=2e-4, lora_r=8, lora_alpha=16, dropout=0.05, max_seq_length=1024. target_modules = TARGET_MODULES sabiti [q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj] — **lm_head/embed YOK** (Qwen3 tied-embeddings; GGUF uyumu). dtype varsayılan fp32, ACHILLES_TRAIN_DTYPE=bf16 ile yarıya iner (AVX512-BF16 olmayan CPU'da emüle edilir → uyarı).
- **YENİ (v1.2):** LoraConfig ve TrainingArguments artık **saf builder'lardan** kurulur (build_lora_kwargs / build_training_kwargs — torch/peft import etmez → çevrimdışı test edilebilir). Yerel trainer bulut reçetesiyle **hızalandı**: artık weight_decay=0.01, warmup_ratio=0.03, lr_scheduler_type="cosine", max_grad_norm=1.0, seed=42 (önceden warmup/scheduler/weight_decay yoktu — degenerasyon/unutma riski). İleri teknikler (rsLoRA/DoRA/init/LoRA+/NEFTune) config alanlarıyla **opt-in** açılır; ayrıntı **Aşama 8**. Profiller load_lora_profile() + achilles train --profile <ad> ile uygulanır.

3c. MLX Eğitimi (macOS ARM)

- **NE:** mlx_lm lora alt-süreciyle eğitim; faz (1–4) resume desteği.
- **NASIL:** build_command() → python -m mlx_lm lora --train --data <dir> --iters --batch-size --num-layers --adapter-path ...; from_phase>0 ise --resume-adapter-file. Bitince AdapterRegistry().register().
- **DOSYA:** app/training/mlx_lora_train.py:24–108.
- **PARAMETRELER:** iterations=300, batch_size=2 (8GB güvenli), lr=1e-4, num_layers=8, save_every=0 (sona ertele → OOM önleme).

3d. Detached (tek-tık) Eğitim

- **NE:** Web/terminal kapansa da eğitimin sürmesi için DETACHED OS süreci.
- **NASIL:** launch() — adapter adı regex doğrulaması (^[A-Za-z0-9_]{1,64}\$), atomik kilit (os.open(O_CREAT|O_EXCL), bayat kilit >120sn temizlenir), ensure_train_split() (clobber onarımı), subprocess.Popen(..., creationflags=DETACHED_PROCESS|CREATE_NEW_PROCESS_GROUP|CREATE_NO_WINDOW) Windows'ta / start_new_session=True POSIX'te. stdout→logs/train-full.log, stderr→logs/train-full-err.log. İlerleme tqdm satırından parse edilir ((\d+)/(\d+)\s*<[...<eta), log <45dk yeni ise "canlı" sayılır.
- **DOSYA:** app/training/detached_launch.py:51–359.
- **HAZIRLIK:** MIN_READY_EXAMPLES=200 (data/lora_sft/lora_sft.jsonl satır sayısı).
- **CLOBBER ONARIMI:** ensure_train_split() — birleşik kaynak doluysa HER ZAMAN seed=42 ile yeniden böler (valid_ratio=0.05); kaynak boşsa mevcut train.jsonl'e dokunmaz.

Aşama 4 — LoRA Profilleri

- **NE:** Hazır eğitim ön-ayarları. **DOSYA:** configs/lora/lora_profiles.yaml.
- **YENİ (v1.2):** Profiller artık **koda bağlı** — load_lora_profile(name) YAML'i PeftTrainConfig alanlarına çevirir; achilles train --profile <ad> uygular. Yeni alanlar: use_rslora, use_dora, init_lora_weights, loraplus_lr_ratio, neftune_noise_alpha, weight_decay, warmup_ratio, lr_scheduler_type, max_grad_norm.

Profil	r	alpha	dropout	epochs	lr	ileri teknik	not
small_smoke_test	8	16	0.05	1	—	—	max_exa pipeline c
standard_reasoning	16	32	0.05	2	2e-4	warmup-cosine-wd	varsayıla
high_capacity_reasoning	32	64	0.05	3	2e-4	rsLoRA + warmup-cosine-wd	büyük ve rsLoRA ç stabil)
discipline_safe	16	32	0.1	1	1e-4	NEFTune(5) + grad-clip + warmup-cosine-wd	v5 reçete lr + az ep güçlü reg catastrop forgetting döngüsü

Aşama 5 — Değerlendirme (adapter_eval)

- **NE:** Eski `ModelEvaluator` base Ollama'yı ölçüyordu, adapter yüklemiyordu. Yeni `adapter_eval`, `transformers/PEFT` ile **gerçek** inference yapar ve **base vs adapter** yan-yana kıyaslar.
- **NASIL:** `_load_model(base, adapter_dir=None)` ile önce BASE, sonra `PeftModel.from_pretrained` ile ADAPTER; her ikisi `model.eval()`, `greedy (do_sample=False, deterministik)`, `max_new_tokens=220`. Her cevapta:
 - **red-flag** (`check_flags`): fiilen yalnız iki deseni uygular — `guaranteed_profit` ve `ignores_costs` (ikincisi **yalnız** cevapta strateji/strategy geçiyorsa tetiklenir, `evaluate_model.py:84-87`) — artı soru-bazlı `must_avoid` token'ları. **NOT:** `success_without_test` deseni `RED_FLAGS` sözlüğünde tanımlıdır (`evaluate_model.py:32-34`) ancak `check_flags` veya başka hiçbir yerde **kullanılmaz** — ölü koddur (`grep` ile doğrulandı: tek geçtiği yer tanım satırı). Bu doküman onu aktif bir red-flag olarak listelemez.
 - **degenerasyon** (`_is_degenerate`): **v1.2'de güçlendirildi** — üç sinyalden biri yeterli: (1) aynı cümlelin tekrarı, (2) aynı 3-gram'ın ≥ 4 kez tekrarı (`_max_ngram_repeat`; nokta ayırıcı olmadan da yakalar — v5 adapter aynı ifadeyi 5 kez yazmıştı), (3) aynı satırın (madde/ liste) tekrarı. Eşikler muhafazakâr → sağlam cevabı yanlış-flag'lemez. Puan: `score = 1.0 - flags/n`; `verdict = accept | reject | inconclusive`.
- **DOSYA:** `app/training/adapter_eval.py:1-156`, `app/training/evaluate_model.py:28-91` (`check_flags:80-91`), CLI `app/main.py:339-374` (`lora-eval`).
- **MODEL/KÜTÜPHANE:** `transformers` (`AutoTokenizer`, `AutoModelForCausalLM`), `peft` (`PeftModel`), `torch` (`bfloat16`, `no_grad`).
- **EVAL SETİ:** `configs/eval/lora_eval_questions.yaml` (~50 soru, 6 alan, seviye 0–4). `YAML'deki scoring_rubric/eval_thresholds` **kodda kullanılmaz** — yalnız `must_avoid` token kontrolü yapılır (**kodda bulunamadı**: rubric değerlendirme).

Aşama 6 — Adapter Registry Yaşam Döngüsü

- **NE:** Adapter durumunu izle; production'a geçiş **kullanıcı onayı** gerektirir.

- **NASIL:** AdapterStatus enum: CANDIDATE → REJECTED | SMOKE_PASSED → EVAL_PASSED → APPROVED → PRODUCTION. JSONL append-only + tam yeniden-yazım (registry/adapters/registry.jsonl). promote(adapter_id, user_approved) — user_approved=False ise hemen döner; mevcut production APPROVED'a indirilir (tek production garantisi), hedef PRODUCTION + approved_by_user=True.
- **DOSYA:** app/lora/adapter_registry.py:20-160, test tests/test_lora_adapter_registry.py:37-63.
- **NOT:** İkinci bir registry (app/training/adapter_registry.py, SQLite + .meta.json sidecar) vardır; kaynak bulgusu bunu **eski/kullanımda olmayan** olarak işaretler (yaşam döngüsü ve testler JSONL versiyonda).

Aşama 7 — Bulut + Aşamalı Eğitim (Stage 1/2)

- **NE:** Yerel CPU'da ≥ 1000 örnek eğitimi haftalar sürer ve overfit eder. Eşik aşıldığında eğitim ücretsiz bulut-GPU'ya (Kaggle T4x2 / Colab T4) taşınır.

Aspekt	Stage 1 (yerel CPU)	Stage 2 (bulut GPU)
İş	sentetik QA üret	gerçek LoRA fine-tune
Süre	sürekli	~30-90 dk
Çıktı	synthetic_qa.jsonl	GGUF Q4_K_M → Ollama

- **GATE 0 (nicelik):** lora-readiness [--threshold 1000] — sentetik satır + onaylı kart örneği toplamı $\geq$ eşik. app/main.py:1715-1767.
- **GATE 1-7 (audit):** lora-audit → control_plane. **KARAR:** ≥ 1000 örnek + audit geçti + kullanıcı onayı.
- **HAZIRLIK KOMUTU:** lora-cloud-prep — birleşik dataset + dedup → lora_sft.jsonl; Unsloth notebook (build_stage2_notebook, placeholder doldurma) → notebooks/achilles_lora_stage2.ipynb; Ollama Modelfile (write_modelfile) → ChatML TEMPLATE + `<|im_start|><|im_end|>` stop token'ları. app/main.py:1770-1849, app/training/cloud_notebook.py:23-66.
- **5 BİLİNEN HATA DÜZELTMESİ (notebook):** (1) 7 target_module, lm_head/embed yok (tied); (2) {"messages": [...]} formatı; (3) apply_chat_template("qwen3-instruct"); (4) dinamik padding + train_on_responses_only (loss maskeleye); (5) save_pretrained_gguf("q4_k_m") + fallback (16-bit merge → llama.cpp).
- **YENİ (v1.2) — parametrik ileri teknikler:** notebook artık {LORA_ALPHA}, {LORA_DROPOUT}, {USE_RSLORA}, {NEFTUNE_ALPHA}, {WEIGHT_DECAY}, {WARMUP_RATIO} placeholder'larını da doldurur (build_stage2_notebook parametreleri config'ten alır). alpha verilmezse 2^*r konvansiyonu. NEFTune SFTConfig(neftune_noise_alpha=...) ile, rsLoRA get_peft_model(use_rslora=...) ile bağlanır. Hepsi **GGUF-güvenli** (eğitim-zamanı / ölçek; mimari değişmez).
- **KRİTİK EŞLEŞMELER:** base = Qwen/Qwen3-4B-Instruct-2507 (Ollama qwen3:4b-instruct-2507 ile birebir; çıplak qwen3:4b=2504 ile uyuşmaz); T4'te fp16 (bf16 DEĞİL — NaN); eğitim chat template ↔ Modelfile TEMPLATE birebir.
- **DOKÜMAN:** docs/PROTOKOL_ASAMALI_EGITIM.md, docs/PROTOKOL_BULUT_EGITIM.md.

- **NOT:** `app/training/cloud_notebook.py` **mevcuttur** (2379 bayt) ve `build_stage2_notebook` (:23) ile `write_modelfile` (:57) fonksiyonlarını içerir; `app/training/peft_lora_train.py:280` içinde `from app.training.cloud_notebook import build_stage2_notebook,` `write_modelfile` ile import edilerek aktif olarak kullanılır (`peft_lora_train.generate_colab_notebook` tarafından çağrılır). Önceki "çelişki/belirsizlik" notu hatalıydı; tek ve net bir durum vardır.

Aşama 8 — İleri LoRA Teknikleri (araştırma entegrasyonu, v1.2)

Bu aşama, 2024–2025 LoRA/SFT literatüründen doğrulanmış ve PEFT 0.19.1 / transformers 5.12.0 / Unsloth tarafından desteklenen iyileştirmeleri Achilles hattına bağlar. Tasarım ilkesi: **hepsi opt-in** (varsayılan davranış değişmez), **GGUF-güvenli** (embedding/lm_head eğitilmez), **deterministik** ve **çevrimdışı-test edilebilir** (saf config builder'lar). Birincil hedef, **v5 catastrophic-forgetting/ degenerasyon regresyonunu** önlemek.

8.1 Ölçek & mimari teknikleri

Teknik	Ne yapar	PEFT/Unsloth	Achilles'te
rsLoRA (rank-stabilized)	Ölçeği α/r yerine $\alpha/\sqrt{r}$ yapar $\rightarrow$ yüksek r 'de gradyan büyümesini dengeler, daha stabil öğrenir	<code>LoraConfig(use_rslora=True)</code>	<code>cfg.use_rslora;</code> <code>high_capacity_re:</code> profilinde açık
DoRA (weight-decomposed)	Ağırlığı büyüklik + yöne ayırır; yalnız yönü LoRA ile öğrenir $\rightarrow$ düşük r 'de tam fine-tune'a yakın	<code>LoraConfig(use_dora=True)</code>	<code>cfg.use_dora (opt-i</code>

init stratejisi	Adapter'ı sıfır yerine bilgilili başlatır: PiSSA (base'in temel bileşenleri), OLoRA (QR), EVA (SVD-aktivasyon), LoftQ (kuantizasyon-farkında), gaussian	<code>LoraConfig(init_lora_weights=...)</code>	<code>cfg.init_lora_weights</code> (varsayılan <code>true</code>)
LoRA+	B matrisine A'dan λ kat daha yüksek lr → daha iyi/hızlı yakınsama (ek parametre yok)	<code>peft.optimizers.create_loraplus_optimizer</code>	<code>cfg.loraplus_lr</code> : ise Trainer'a özel opt

`build_lora_kwargs(cfg)` bu seçenekleri saf bir sözlüğe çevirir (torch import etmeden) →
`LoraConfig(**build_lora_kwargs(cfg)).normalize_init_lora_weights()` "true"→True,
"pissa"→"pissa" eşler ve bilinmeyen değerde **erken ValueError** verir (sessiz yanlış-init yerine).

8.2 Regularizasyon & unutmaya-karşıtı (v5 dersinin doğrudan yanıtı)

- **NEFTune** (noisy embedding fine-tuning): eğitimde embedding'lere ölçekli rastgele gürültü ekler → ezber/overfit azalır, yönerge-takibi davranışı **korunur**.
`TrainingArguments(neftune_noise_alpha= $\alpha$ )` (transformers yerel desteği).
`cfg.neftune_noise_alpha>0` açar; 0.0 → anahtar hiç eklenmez (hook açılmaz).
`discipline_safe`'te $\alpha=5$.
- **Düşük lr + az epoch**: instruct modelini bozmamanın en güçlü kaldırıcısı. `discipline_safe`:
`lr=1e-4` (2e-4 yerine), `epochs=1`. Aşırı epoch = unutmaya + tekrar döngüsü.
- **Yüksek dropout (0.1)**: küçük veri setinde overfit'i bastırır.
- **Warmup + cosine + weight_decay + grad-clip**: yerel trainer artık bunları her zaman uygular (`build_training_kwargs`); ani gradyan sıçramaları (degenerasyon tetikleyicisi)
`max_grad_norm=1.0` ile kırılır.
- **Veri tarafı (replay/rehearsal)**: `discipline_dataset.py` adversarial disiplin örneklerini `mix_discipline()` ile ~%25 karıştırır → model yalnız "pasaja göre cevapla" öğrenip refusal/disiplin yeteneğini **unutmaz**. Bu, literatürdeki rehearsal/replay ilkesinin Achilles uygulamasıdır.

8.3 v5 catastrophic-forgetting reçetesi (`discipline_safe`)

v5 adapter base'den **daha kötü** oldu: garanti-kâr reddini bıraktı, maliyet uydurdu, aynı ifadeyi 5 kez tekrarladı (degenerate). Önerilen kombine reçete (kanıt biriktikçe ayarlanır):

```
profil: discipline_safe (achilles train --profile discipline_safe)
r=16, alpha=32 (2r), dropout=0.1, epochs=1
lr=1e-4, scheduler=cosine, warmup=0.05, weight_decay=0.01, max_grad_norm=1.0
NEFTune  $\alpha=5$ 
veri: synth-QA + discipline_dataset ~%25 (lora-cloud-prep --discipline-ratio 0.25)
rsLoRA=kapalı (muhafazakâr; lr ile birlikte ayarlandı)
```

Doğrulama zorunlu (CLAUDE.md Kural 2): Bu reçete *hipotez* tir. Sıra: (1) dataset'i OFFLINE sınavdan geçir (L3/L4/L5 + understanding-score, **eğitMEDEN**), (2) bulutta eğit, (3) adapter_eval ile base-vs-adapter karşılaştır, (4) yalnız adapter base'i **geçerse** terfi. Aynı veriyle körlemesine retrain **yasak** (overfit).

8.4 Değerlendirilen ama şimdilik entegre EDİLMEYEN

- **QLoRA (4-bit) yerelde:** bulut notebook'unda zaten load_in_4bit=True; yerel CPU'da bnb kuantizasyonu pratik değil → yalnız bulutta.
- **PiSSA/OLoRA varsayılan açma:** GGUF residual dönüşümü ek karmaşıklık → opt-in bırakıldı, dokümanite edildi.
- **DPO/ORPO (tercih optimizasyonu):** auto_researcher reward verisi üretir ama SFT sonrası ayrı bir aşama; bu turda kapsamadı (gelecek tur).

Kullanılan Loop'lar ve Otomasyon

Loop / Mekanizma	Tetikleyici	Ne yapar
scripts/continuous-learning.sh	Manuel (CLI)	kart → onay → anlama → (3 turda bir) research+synth → synth-qa → rag-mastery; başta STOP_TRAINING ile eski eğitimi devralır
scripts/auto-chain.sh	Manuel	7 aşama: kart→onay→anlama→research→synth-paper→lora-dataset→ 24h eğitim döngüsü (train --run --backend peft --iterations 40)
scripts/mac-loop.sh	Manuel (macOS)	tur: kart→onay→synth-qa→lora-dataset→ MLX eğitim (300 iter); storage/train_status.json ile web rozeti güncelle
app/pipeline/auto_researcher.py	achilles auto-research	onaylı kart → hipotez sorusu → tool-use seans → reward skorla (DPO hazırlığı)

app/training/detached_launch.py	Web POST /api/training/launch / "EĞİTİME HAZIR" butonu	tek eğitim başlatma + atomik kilit + clobber onarımı + log-tail status
AutoLoRAPipeline.background_loop	auto_enabled=True (varsayılan False)	her aralıkta IDLE/GATE_FAILED ise check_and_prepare() (Gate 0–8)

ScheduleWakeup notu: Direktif "OS-süreci mi ScheduleWakeup mı" ayrımını ister.

Kaynak bulgularında **ScheduleWakeup tabanlı bir nöbet mekanizması kodda bulunamadı**; HANDOFF notu da "yeni seansta kendiliğinden devam ETMEZ" der.

Tüm loop'lar ya manuel bash, ya web-tetikli detached subprocess, ya da web-içi asyncio'dur. Repo kökündeki scripts/achilles-autostart.vbs / achilles-loop-autostart.vbs (git status'ta izlenmemiş) Windows autostart için olabilir ancak içerikleri bu bulgularda yer almadığından **doğrulanmadı**.

Kararlar ve Dersler

- 1. Kaba yüzde yerine objektif sınav.** "Anlama %X" gibi öz-değerlendirme yerine, model SINAVLA (L1/L2 RAG sadakati; L3 sayısal np.allclose; L4 karşıolgu yön; L5 kompozisyon 3-kapı) ölçülür. LLM yoksa skipped/no_data — **sahte pass üretilmez** (understanding_score.py:152–174, l3_application.py:89–100). Bu, CLAUDE.md Kural 2'nin (test edilmeden "çalışıyor" deme) doğrudan uygulamasıdır.
- 2. v5 regresyon dersi.** v5 eğitimi tamamlandı ancak disiplinde GERİLEDİ. reports/evals/adapter_eval_achilles_lora_v5_discipline_core.json: adapter aynı ifadeyi 5 kez tekrarladı (degenerate_repetition), skorlar negatif (base=-2.0, adapter=-1.0), buna rağmen verdict accept. **Ders:** (a) degenerasyon cezası skoru daha ağır etkilemeli; (b) üretim öncesi manuel inceleme zorunlu; (c) eval harness'in adapter'ı gerçekten yüklemesi şart — bu yüzden adapter_eval yazıldı.
- 3. Negasyon-kör (negation-blind) uyarısı.** check_flags() basit string eşleşmesidir (token.lower() in answer.lower(), evaluate_model.py:88–89) **ve** guaranteed_profit/ignores_costs desenleri de regex temellidir. "Kesinlikle **değil** kazanç" gibi olumsuzlamalar yanlış-pozitif bayrak tetikleyebilir. Negasyon-farkında kontrol **kodda bulunamadı** (evaluate_model.py:82–91).
- 4. Sürekli CPU-LoRA durduruldu, RAG-first + sentetik veri motoru.** 4B model CPU'da ~74–76 sn/adım; 15–50 örnek overfit. Karar: önce robust RAG + grounding'li sentetik QA ile ≥1000 örneğe ulaş, sonra bulut-GPU. (MEMORY: rag-training-redesign, v5-adapter-regression.)
- 5. eval/exec yasağı, whitelist AST.** Makaleden çıkan formüller safe_eval ile yalnız beyaz-listeli AST düğümleriyle yürütülür; L3/L4/L5 parse'ları yalnız JSON. (safe_eval.py:44–92.) CLAUDE.md Kural 5.
- 6. Determinizm her yerde.** Split seed=42, synth-qa seed+persona_index, synthetic_closes(seed), greedy inference. CLAUDE.md Kural 6.

7. **Production terfi yalnız kullanıcı onayıyla.** `promote(..., user_approved=True)` ve eval seti yoksa `EVAL_SKIPPED` → terfi bloklanır. CLAUDE.md Kural 8 + "Anayasa II/VI".
8. **Clobber onarımı ve atomik kilit.** Detached eğitim sırasında kaynak güncellenirse train/valid'in 0'a düşmesi `ensure_train_split()` ile her başlatmada onarılır; çift-tık atomik kilitle engellenir.
9. **Tied-embedding tuzağı.** Qwen3 tied-embeddings nedeniyle `lm_head/embed` delta'ları GGUF'ta sessizce atlanır → adapter bozulur. Çözüm: yalnız 7 attention+MLP modülü hedeflenir.

Dosya Referans Haritası

Dosya	Görev
<code>app/lora/dataset_builder.py</code>	Onaylı kart → LoRA SFT örneği (<code>card_to_lora_example</code> , <code>_build_answer</code>)
<code>app/brain/synthetic_qa_builder.py</code>	Sentetik QA üretimi, grounding (RAFT-vari), <code>dedup_jsonl_lines</code>
<code>app/brain/knowledge_card_builder.py</code>	PDF → yapılandırılmış bilgi kartı (LLM, JSON, draft)
<code>app/research/chain_data_builder.py</code>	Araştırma oturumu → reasoning-chain eğitim verisi
<code>app/training/unified_dataset.py</code>	Kart + mastery + tool-use birleşik MLX dataset
<code>app/lora/quality_filter.py</code>	Gate 4: uzunluk/örtüşme/duplicate (SHA256)
<code>app/lora/dataset_splitter.py</code>	Gate 8: source-bazlı train/valid/test + leakage
<code>app/lora/curriculum.py</code>	Müfredat seviyeleri 0–4 (<code>classify_curriculum(card_json, difficulty)</code>)
<code>app/lora/gates.py</code>	Gate 0–8 saf fonksiyonları
<code>app/lora/safety_scanner.py</code>	Gate 7 BLOCKER: sır/PII/finansal yönlendirme regex
<code>app/lora/math_verifier.py</code>	Gate 5: istatistik bias + aşırı-emin dil
<code>app/lora/domain_classifier.py</code>	Gate 0/3: anahtar-kelime domain sınıflandırma
<code>app/lora/control_plane.py</code>	Gate orkestrasyonu (<code>run_audit/run_full</code>), rapor
<code>app/lora/auto_pipeline.py</code>	Durum makinesi + <code>background_loop</code> + eval/promote akışı
<code>app/training/backend.py</code>	MLX vs PEFT platform tespiti
<code>app/training/peft_lora_train.py</code>	PEFT eğitimi (<code>dry-run/--run</code>), loss eğrisi; saf builder'lar <code>build_lora_kwargs/build_training_kwargs/</code> <code>recipe_summary</code> , <code>load_lora_profile</code> , <code>normalize_init_lora_weights</code> , ileri teknikler (rsLoRA/ DoRA/init/LoRA+/NEFTune)
<code>app/training/mlx_lora_train.py</code>	MLX eğitimi, faz resume

app/training/detached_launch.py	Detached subprocess başlatma + status + clobber onarımı
app/training/adapter_eval.py	Base-vs-adapter gerçek PEFT değerlendirme
app/training/evaluate_model.py	Red-flag kontrolleri (check_flags); success_without_test deseni tanımlı ama kullanılmaz (ölü kod)
app/lora/adapter_registry.py	JSONL yaşam döngüsü + promote (kullanıcı onayı)
app/training/adapter_registry.py	(Eski) SQLite + sidecar registry
app/training/cloud_notebook.py	Stage 2 notebook + Modelfile üretimi (build_stage2_notebook, write_modelfile); peft_lora_train tarafından aktif kullanılır
configs/lora/lora_profiles.yaml	LoRA ön-ayar profilleri (r/alpha/epochs + ileri teknik alanları); discipline_safe v5 reçetesi; load_lora_profile ile koda bağlı
docs/egitim/ LORA_ARASTIRMA_LOG.md	Tekrarlı araştırma logu + dedup defteri (kapsanan teknik/ kaynak)
tests/test_peft_lora_recipe.py	İleri reçete builder'larının çevrimdışı regresyon testleri
tests/ test_adapter_eval_degenerate.py	Güçlendirilmiş degenerasyon (n-gram/satır döngüsü) testleri
configs/eval/ lora_eval_questions.yaml	~50 soruluk disiplin eval seti
app/verification/exams/ understanding_score.py	L1–L5 sınav agregasyonu (objektif skor)
app/verification/exams/ safe_eval.py	Whitelist AST (eval/exec yok)
scripts/continuous-learning.sh	72h kart/anlama/synth-qa loop
scripts/auto-chain.sh	Tek-zincir + 24h eğitim loop
scripts/mac-loop.sh	macOS MLX eğitim loop
app/pipeline/auto_researcher.py	Kart → soru → tool-use → reward
app/main.py	CLI: train, lora-eval, lora-readiness, lora-audit, lora-cloud-prep, synth-qa (-bulk), auto-research
storage/auto_lora_state.json	Durum makinesi kalıcılığı

Bilinen Sınırlamalar

- Sistem son durumu train_failed.** storage/auto_lora_state.json örneğinde Gate geçilmiş ama eğitim tamamlanamamış. Hattın baştan-sona başarılı bir koşusu bu bulgularda **kanıtlanmadı**.
- RAFT modülü yok.** Grounding RAFT ilkesini uygular ama bağımsız "RAFT seed" mekanizması **kodda bulunamadı**; storage/_genRAFT_seed.py izlenmemiş/doğrulanmamış.

3. **Negasyon-kör red-flag.** Basit string/regex eşleşmesi olumsuzlamaları ayırt edemez.
4. **Degenerasyon cezası hafif (kısmen giderildi).** v1.2'de **tespit** güçlendirildi (cümle + n-gram + satır döngüsü; `_max_ngram_repeat`). Ancak ceza hâlâ tek bir bayrak ağırlığındadır; "tek degenerasyon → koşulsuz reject" override henüz eklenmedi (skor + regression kontrolü dolaylı yakalar).
5. **Inference hata yönetimi yok.** `adapter_eval`'de model yükleme/inference başarısızlığında `try/except` **kodda bulunamadı** → crash.
6. **Eval rubric kullanılmaz.** YAML'deki `scoring_rubric/eval_thresholds` okunmaz; yalnız `must_avoid` token kontrolü.
7. **Cross-batch duplicate izleme yok.** Gate 4 yalnız oturum-içi hash; kalıcı DB log yok.
8. **Müfredat pacing / domain balance yok.** Gate 2/3 yalnız aralık/varlık kontrolü; %60/%30/%10 dağılım ve hedef domain dengesi **kodda bulunamadı**.
9. **Ölü red-flag deseni.** `success_without_test` deseni `RED_FLAGS`'te tanımlıdır (`evaluate_model.py:32-34`) ama `check_flags` veya başka hiçbir yerde çağrılmaz; etkin değildir. (Aktif desenler yalnız `guaranteed_profit` ve strateji-kışullu `ignores_costs + must_avoid`.)
10. **Çift registry.** JSONL (aktif) ve SQLite (eski) iki registry; karışıklık riski.
11. **MasterySFTBuilder / build_tool_use_dataset / ModelEvaluator / LoRAControlPlane.run_full** iç detayı bu bulgularda **tam okunmadı** → davranışları doğrulanmadı.
12. **CPU'da eğitim pratik değil.** 4B model ~74–76 sn/adım; ≥1000 örnek için bulut-GPU zorunlu.
13. `auto_researcher` → **tool-use** zinciri DPO hazırlığı üretir; DPO eğitiminin kendisi (eğitim döngüsüne bağlanması) bu bulgularda **gösterilmedi**.
14. **v1.2 reçetesi (discipline_safe) henüz doğrulanmadı.** İleri teknikler kod+config olarak bağlandı ve çevrimdışı test edildi; ancak bir bulut eğitim koşusu + `adapter_eval` gate'i ile **fiilen daha iyi olduğu kanıtlanmadı** (Kural 2: test edilmeden "daha iyi" denmez). `rsLoRA/DoRA/init/LoRA+/NEFTune` varsayılan kapalı (opt-in) — yalnız regularizasyon (`warmup/cosine/weight_decay/grad-clip`) varsayılan etkin.

Araştırma Kaynakları & Tekrarlı Tarama

Tekrarlı araştırma döngüsü (2026-06-17 kararı)

LoRA literatürü sürekli ilerlediği için Achilles, yenilikleri periyodik tarar ve **işe yarayanları** entegre eder. Tasarım (kullanıcı kararı):

- **Günlük hafif tarama:** `arXiv/HF/Unsloth/PEFT`'te `docs/egitim/LORA_ARASTIRMA_LOG.md` defterinde **olmayan** yeni öge var mı? Yoksa no-op. Varsa → loga aday + doküman auto-push.
- **Haftalık derin tur:** tam çok-ajanlı sweep (paralel arama → adversarial doğrulama → sentez) + entegrasyon.
- **Kapı:** kod/reçete değişikliği `main`'e körlemesine push **edilmez** → PR / inceleme (doğrulanmamış reçete riski; CLAUDE.md Kural 2/8). Doküman-yalnız güncelleme auto-push olabilir.
- **Defter:** her tur `LORA_ARASTIRMA_LOG.md`'ye yazılır; "Kapsanan teknikler/kaynaklar" listesi dedup anahtardır (aynı şey iki kez derin-arastırılmaz).

Bu sürümün (v1.2) kaynakları — Tur 1, 2026-06-17

Aşağıdaki kaynaklar çok-ajanlı araştırma turunda **gerçekten getirilip** adversarial doğrulandı (uydurma değil; bağlantısı doğrulanamayan teknikler entegre edilmedi).

Teknik / Konu	Kaynak
LoRA hiperparametre rehberi (rank/alpha/lr/epoch)	Unsloth — LoRA Hyperparameters Guide: https://unsloth.ai/docs/get-started/fine-tuning-llms-guide/lora-hyperparameters-guide
Qwen3 fine-tune (chat template, target modules)	Unsloth — Qwen3 tutorial: https://unsloth.ai/docs/models/tutorials/qwen3-how-to-run-and-fine-tune
rsLoRA / DoRA / init (PiSSA/OLoRA/EVA/LoftQ) / LoRA+ — API	HF PEFT — LoRA developer guide: https://huggingface.co/docs/peft/main/en/developer_guides/lora
NEFTune / response-only / packing — SFT	HF TRL — SFTTrainer: https://huggingface.co/docs/trl/sft_trainer
rsLoRA (rank-stabilized scaling)	arXiv 2312.03732
DoRA (weight-decomposed)	arXiv 2402.09353 · https://github.com/NVlabs/DoRA
NEFTune (noisy embedding FT)	arXiv 2310.05914 · https://github.com/neelsjain/NEFTune
LoftQ (kuantizasyon-farkında init)	https://github.com/yxli2123/LoftQ
Nöral metin degenerasyonu (tekrar/repetition)	arXiv 1904.09751

Tam ve güncellenen liste: docs/egitim/LORA_ARASTIRMA_LOG.md.

Sözlük

- **LoRA (Low-Rank Adaptation):** Tam modeli değil, eklenen düşük-ranklı (r) matrisleri eğiterek ucuz fine-tune.
- **SFT (Supervised Fine-Tuning):** Etiketli (soru→cevap / messages) örneklerle denetimli ince-ayar.
- **PEFT:** Parameter-Efficient Fine-Tuning kütüphanesi (torch+transformers); Windows/Linux yolu.
- **MLX:** Apple Silicon için ML çerçevesi; mlx_lm lora ile eğitim.
- **Grounding:** Cevabın yalnız verilen pasaja dayanması; uydurma metrik/sayı reddi.
- **RAFT:** Retrieval-Augmented Fine-Tuning; "yalnız bağlamdan cevapla" ilkesi (burada grounding olarak uygulanır).
- **Gate 0–8:** Sıralı kalite kapıları; bazıları BLOK, biri (7) mutlak BLOCKER, ikisi (5/6) UYARI.
- **BLOCKER:** Tek ihlalin tüm batch'i reddettiği kapı (Gate 7 güvenlik).
- **Control Plane:** Kapı orkestrasyonu + durum makinesi (IDLE...PROMOTED).
- **Detached süreç:** Web/terminal kapansa da süren bağımsız OS alt-süreci.
- **Clobber onarımı:** train/valid'in 0'a düşmesini her başlatmada kaynaktan yeniden bölerek önleme.
- **Dry-run:** Eğitimi başlatmadan bağımlılık/komut doğrulaması (--run olmadan varsayılan).
- **Degenerasyon:** Adapter'ın aynı cümleyi tekrarlaması (overfit/çöküş işareti).
- **Adapter registry:** Adapter durum + metadata defteri (JSONL); production'a terfi kullanıcı onaylı.
- **GGUF / Q4_K_M:** llama.cpp/Ollama model formatı / 4-bit k-quant kvantizasyon.

- **Modelfile:** Ollama model tanımı (FROM + TEMPLATE + SYSTEM + stop token).
- **Tied embeddings:** Giriş/çıkış embedding'lerinin paylaşıldığı mimari; GGUF'ta lm_head delta'sı atlanır.
- **Stage 1/2:** Yerel sentetik veri üretimi (1) → bulut-GPU LoRA eğitimi (2).
- **UnderstandingScore (L1–L5):** Anlamayı yüzdeyle değil sınavla kanıtlayan objektif skor.
- **source_id:** Örneğin kaynak makalesi; train/test sızıntısını önlemek için bölme anahtarı.
- **Ölü kod:** Tanımlı ama hiçbir çağrı yolunda kullanılmayan kod (örn. success_without_test deseni).
- **rsLoRA (rank-stabilized LoRA):** Ölçeği α/r yerine $\alpha/\sqrt{r}$ yapan düzeltme; yüksek rank'ta öğrenmeyi stabilize eder.
- **DoRA (weight-decomposed LoRA):** Ağırlığı büyüklük + yön bileşenine ayırıp yalnız yönü LoRA ile öğrenen yöntem; düşük rank'ta kaliteyi artırır.
- **LoRA+:** A ve B düşük-rank matrislerine farklı (B'ye daha yüksek) öğrenme oranı veren optimizier ayarı.
- **init_lora_weights (PiSSA/OLoRA/EVA/LoftQ/CorDA):** Adapter'ı sıfır yerine base'in yapısından bilgili başlatma stratejileri; hızlı yakınsama. PiSSA/OLoRA/CorDA base'i değiştirir → GGUF için residual dönüşümü gerekir.
- **NEFTune:** Eğitimde embedding'lere ölçekli gürültü ekleyerek overfit'i azaltıp yönerge-takibini koruyan regularizasyon.
- **Catastrophic forgetting:** Fine-tune sırasında modelin önceki yeteneklerini/davranışını (örn. refusal/disiplin) yitirmesi; v5 regresyonunun kökü.
- **Rehearsal/replay:** Unutmayı önlemek için eğitime önceki davranışı temsil eden örnekler (Achilles'te adversarial disiplin örnekleri) karıştırmak.